Programmazione 1 in C

Esercizi di ripasso (prima prova intermedia)

10 coppie base e avanzata · Lezioni 1–8

Questa raccolta propone esercizi facoltativi sugli argomenti svolti fino alla lezione 8. Ogni coppia contiene due programmi distinti: completa la versione base e poi affronta quella avanzata, rispettandone i nuovi requisiti. Le coppie seguono una progressione complessiva di difficoltà.

Indicazioni generali

Strumenti consentiti. Usa tipi primitivi, operatori, condizioni, `switch`, cicli, funzioni e array monodimensionali. Sono consentite le funzioni `printf`, `scanf`, `rand` e `srand` delle librerie `stdio.h` e `stdlib.h`.

Input. Puoi assumere che tutti i dati siano validi, del tipo richiesto e negli intervalli indicati. Non sono necessari controlli di validità. Tutti gli intervalli includono gli estremi; spazi e a capo possono separare i dati.

Array e caratteri. Per gli array usa una capacità costante `MAX` di valore 20. La lunghezza effettiva è il numero di elementi indicato nell'esercizio: usa soltanto queste posizioni. I caratteri seguono la codifica ASCII.

Funzioni. Quando richieste, implementa e usa le funzioni descritte, con parametri e tipo di ritorno appropriati. Lettura e stampa spettano al `main`; le funzioni ricevono i dati necessari come parametri. Le modifiche richieste agli array devono essere visibili anche dopo il ritorno al `main`. Puoi aggiungere funzioni di supporto.

Output. Stampa soltanto le righe richieste, senza messaggi per chiedere l'input. Inserisci un solo spazio dopo i due punti e tra gli elementi degli array, senza spazi finali. Ogni riga termina con un a capo. I risultati reali devono avere esattamente due cifre decimali.

Esempi. Ogni riga di una tabella rappresenta un'esecuzione indipendente. Nella coppia 08 le tabelle mostrano esiti ammessi per i numeri casuali, senza prevedere la sequenza prodotta dal seme.

Per ogni programma verifica gli esempi proposti e aggiungi casi scelti da te, comprese le situazioni indicate alla fine della coppia.

Percorso della raccolta

Coppie	Argomenti principali
01–03	Operatori, scarti con segno, switch, funzioni, conversioni e cifre
04–07	Estremi, caratteri, modifica degli array, gruppi e simmetrie
08–10	Generazione pseudocasuale, percentuali, alternanze e valori vicini

01 Multipli vicini

Obiettivo. Usare divisione, resto e confronti per descrivere uno scarto con segno.

Leggi due interi: `x`, compreso tra 0 e 200, e `passo`, compreso tra 2 e 20. Considera i multipli non negativi di `passo`, incluso zero. Risolvi entrambe le versioni senza cicli.

A Versione base

Trova il maggiore multiplo di `passo` minore o uguale a `x` e il minore multiplo maggiore o uguale a `x`. Stampa i due risultati sulle righe `Inferiore:` e `Superiore:`. Se `x` è già un multiplo, i due risultati coincidono con `x`.

Input	Output
`23 5`	`Inferiore: 20` `Superiore: 25`
`20 5`	`Inferiore: 20` `Superiore: 20`

B Versione avanzata

Trova il multiplo non negativo di `passo` più vicino a `x`. A parità di distanza scegli quello maggiore.

Stampa `Multiplo:` seguito dal valore scelto e, sulla riga successiva, `Scarto:` seguito da `x` meno il multiplo scelto. Lo scarto può essere negativo, nullo o positivo.

Input	Output
`22 4`	`Multiplo: 24` `Scarto: -2`
`1 4`	`Multiplo: 0` `Scarto: 1`
`30 5`	`Multiplo: 30` `Scarto: 0`

Casi da verificare. Valore zero, multiplo esatto, distanze uguali e casi con scarto positivo o negativo.

02 Misure nella stessa unità

Obiettivo. Combinare switch, parametri, cicli e risultati reali.

Una misura è descritta da un carattere `unita` e da un intero `valore` tra 0 e 1000. I caratteri `C` o `c` indicano centimetri, `D` o `d` decimetri, `M` o `m` metri. Un metro corrisponde a 100 centimetri oppure a 10 decimetri.

Scrivi e usa `in_metri`, che riceve il carattere dell'unità e il valore della misura e restituisce un numero reale: la misura espressa in metri. La funzione deve utilizzare uno `switch`. Non usare array in questa coppia.

A Versione base

Leggi, nell'ordine, `unita` e `valore`. Stampa `Metri:` seguito dalla misura convertita, con due cifre decimali.

Input	Output
`C 125`	`Metri: 1.25`
`d 7`	`Metri: 0.70`

B Versione avanzata

Leggi `n`, intero tra 1 e 10, poi `n` misure, ciascuna nel formato `unita valore`. Usa `in_metri` per ogni misura.

Stampa il totale delle misure convertite sulla riga `Totale:` e la loro media sulla riga `Media:`, entrambi espressi in metri e con due cifre decimali.

Input	Output
`3` `C 125 d 5 M 2`	`Totale: 3.75` `Media: 1.25`
`2` `C 50 D 5`	`Totale: 1.00` `Media: 0.50`
`1 m 0`	`Totale: 0.00` `Media: 0.00`

Casi da verificare. Unità maiuscole e minuscole, valore zero e medie non intere.

03 Bilancio delle cifre

Obiettivo. Visitare le cifre di un intero e distinguere contributi positivi e negativi.

Leggi un intero `n` tra 0 e 30000. Lavora sulle sue cifre decimali senza convertirlo in una stringa.

A Versione base

Scrivi e usa `somma_cifre`, che riceve `n` e restituisce la somma delle sue cifre. Per `n` uguale a zero il risultato è zero. Stampa il risultato dopo `Somma:`.

Input	Output
`1234`	`Somma: 10`
`0`	`Somma: 0`

B Versione avanzata

Scrivi e usa `bilancio_cifre`, che riceve `n` e restituisce un intero ottenuto alternando il segno dei contributi delle cifre: le unità si sommano, le decine si sottraggono, le centinaia si sommano e così via.

Considera soltanto le cifre effettive del numero. Per `n` uguale a zero il risultato è zero. Stampa il risultato dopo `Bilancio:`.

Input	Output
`1234`	`Bilancio: 2`
`1010`	`Bilancio: -2`
`0`	`Bilancio: 0`

Casi da verificare. Numero di una cifra, cifre zero intermedie, numero pari o dispari di cifre e risultato negativo.

04 Livellare le risorse

Obiettivo. Cercare un estremo e calcolare una quantità complessiva senza modificare i dati.

Leggi `n`, intero tra 1 e 20, poi `n` interi tra 0 e 50 e memorizzali nell'array `risorse`. Ogni valore indica quante unità possiede una postazione. Le funzioni di questa coppia non devono modificare l'array.

A Versione base

Scrivi e usa `massimo_valore`, che riceve `n` e l'array e restituisce il maggiore valore presente. Stampa il risultato dopo `Massimo:`.

Input	Output
4 3 7 7 1	`Massimo: 7`
1 0	`Massimo: 0`

B Versione avanzata

Scrivi e usa `costo_livellamento`, che riceve `n` e l'array. Restituisce il numero totale di unità da aggiungere affinché ogni postazione raggiunga il massimo inizialmente presente, senza togliere unità a nessuna postazione.

Ogni postazione partecipa al calcolo, anche se possiede già il massimo. Stampa il risultato dopo `Da aggiungere:`.

Input	Output
4 3 7 7 1	`Da aggiungere: 10`
3 5 5 5	`Da aggiungere: 0`
2 0 10	`Da aggiungere: 10`

Casi da verificare. Una sola postazione, massimo ripetuto, tutti i valori uguali e presenza di zeri.

05 Maiuscole a blocchi

Obiettivo. Usare operazioni sui caratteri e modificare un array secondo le posizioni.

Usa un array di caratteri chiamato `lettere`. Ogni carattere letto è una lettera inglese, maiuscola da `A` a `Z` oppure minuscola da `a` a `z`. Mantieni l'ordine e l'identità alfabetica delle lettere: può cambiare soltanto il loro caso.

Esegui le conversioni mediante operazioni sui caratteri, senza funzioni di libreria. In entrambe le versioni stampa `Lettere:` seguito dall'array finale.

A Versione base

Leggi `n`, intero tra 1 e 20, poi le `n` lettere. Scrivi e usa `tutte_maiuscole`, che riceve `n` e l'array e sostituisce direttamente ogni minuscola con la corrispondente maiuscola. Le lettere già maiuscole restano tali.

Input	Output
3 a Z b	Lettere: A Z B
1 q	Lettere: Q

B Versione avanzata

Leggi, nell'ordine, `n` tra 1 e 20, `k` tra 1 e 5 e poi le `n` lettere. Scrivi e usa `alternanza_blocchi`, che riceve `n`, `k` e l'array e lo modifica direttamente.

Suddividi le posizioni in blocchi consecutivi di `k` elementi. Rendi maiuscole tutte le lettere del primo blocco, minuscole quelle del secondo, maiuscole quelle del terzo e così via. L'ultimo blocco può avere meno di `k` elementi; applica comunque la regola prevista per quel blocco.

Input	Output
7 2 a B c D E F g	Lettere: A B c d E F g
5 1 a B c D e	Lettere: A b C d E
3 5 a b C	Lettere: A B C

Casi da verificare. Lettere già nel caso richiesto, k uguale a 1, blocco finale incompleto e k maggiore di n.

06 Tre posizioni alla volta

Obiettivo. Riordinare gruppi disgiunti conservando tutti i valori.

Leggi `n`, intero tra 1 e 20, poi `n` interi tra -20 e 20 nell'array `v`. Considera le prime tre posizioni come una terna, le successive tre come un'altra terna e così via. Le terne non si sovrappongono.

Se alla fine restano uno o due elementi, lasciali invariati. In entrambe le versioni modifica direttamente l'array e stampalo dopo `Valori:`.

A Versione base

Scrivi e usa `ruota_terne`, che riceve `n` e l'array. Per ogni terna completa trasforma l'ordine dei valori da `a b c` a `b c a`. Puoi usare un array di appoggio.

Input	Output
4 3 1 2 8	`Valori: 1 2 3 8`
5 4 -2 -5 9 0	`Valori: -2 -5 4 9 0`

B Versione avanzata

Scrivi e usa `ruota_terne_condizionate`, che riceve `n` e l'array. Per ogni terna completa considera la somma dei suoi tre valori prima della modifica:

- Se la somma è positiva, trasforma `a b c` in `b c a`.
- Se la somma è negativa, trasforma `a b c` in `c a b`.
- Se la somma è zero, lascia la terna invariata.

Non usare altri array; puoi usare variabili semplici di appoggio.

Input	Output
9 1 2 3 4 -2 -5 1 -1 0	`Valori: 2 3 1 -5 4 -2 1 -1 0`
5 4 -2 -5 9 0	`Valori: -5 4 -2 9 0`
2 4 7	`Valori: 4 7`

Casi da verificare. Meno di tre elementi, terne con somma positiva, negativa o nulla e una coda di uno o due elementi.

07 Simmetrie con segno

Obiettivo. Confrontare posizioni speculari e trattare correttamente l'elemento centrale.

Leggi `n`, intero tra 1 e 20, poi `n` interi tra -20 e 20 nell'array `v`. Le funzioni di questa coppia non devono modificare l'array. Stampa `Esito:` seguito da 1 se la proprietà richiesta è verificata, altrimenti da 0.

A Versione base

Scrivi e usa `speculare`, che riceve `n` e l'array. Restituisce 1 se il primo valore è uguale all'ultimo, il secondo al penultimo e così via; altrimenti restituisce 0.

Con lunghezza dispari il valore centrale può essere qualsiasi. Un array di un solo elemento soddisfa la proprietà.

Input	Output
3 2 7 2	Esito: 1
3 2 7 -2	Esito: 0

B Versione avanzata

Scrivi e usa `speculare_opposto`, che riceve `n` e l'array. Restituisce 1 se il primo valore è l'opposto dell'ultimo, il secondo è l'opposto del penultimo e così via; altrimenti restituisce 0.

Con lunghezza dispari il valore centrale deve essere zero. Anche un array formato da un solo zero soddisfa la proprietà.

Input	Output
5 2 -3 0 3 -2	Esito: 1
5 2 -3 1 3 -2	Esito: 0
1 0	Esito: 1

Casi da verificare. Lunghezza pari o dispari, valori tutti zero, una sola coppia non conforme e valore centrale diverso da zero.

08 Numeri di due cifre

Obiettivo. Generare dati con un vincolo e calcolare una percentuale sui valori memorizzati.

Leggi soltanto `n`, intero tra 1 e 20. Inizializza una sola volta il generatore pseudocasuale con seme 42 nel `main`, prima di generare i numeri. Memorizza i valori nell'array `numeri` e stampa `Valori:` seguito dai primi `n` elementi.

Le tabelle mostrano esiti ammessi dalle regole, non la sequenza prevista dal seme. I valori prodotti da `rand()` possono variare tra librerie C.

A Versione base

Scrivi e usa `genera_due_cifre`, che riceve `n` e l'array da riempire. Genera per ogni elemento un intero compreso tra 10 e 99, effettuando esattamente una chiamata a `rand()` per valore memorizzato. Sono ammesse ripetizioni.

Input	Un esito ammesso
`3`	`Valori: 10 55 99`
`1`	`Valori: 42`

B Versione avanzata

Scrivi e usa `genera_cifre_diverse`, che riceve `n` e l'array da riempire. Genera candidati tra 10 e 99: accetta un candidato soltanto se la cifra delle decine è diversa da quella delle unità. Se le due cifre sono uguali, scarta il candidato e genera di nuovo per la stessa posizione. Sono ammesse ripetizioni di numeri accettati e tutte le chiamate aggiuntive a `rand()` necessarie.

Scrivi e usa anche `percentuale_crescenti`, che riceve `n` e l'array generato e restituisce la percentuale di numeri la cui cifra delle decine è minore di quella delle unità, rispetto ai `n` numeri memorizzati. Non modificare l'array.

Dopo la riga `Valori:`, stampa `Crescenti:` seguito dalla percentuale con due cifre decimali, senza simbolo percentuale.

Input	Un esito ammesso
`4`	`Valori: 24 71 39 50` `Crescenti: 50.00`
`1`	`Valori: 42` `Crescenti: 0.00`
`3`	`Valori: 12 23 89` `Crescenti: 100.00`

Casi da verificare. Una sola estrazione, estremi 10 e 99 nella versione base, cifre uguali da scartare e percentuali zero o cento.

09 Alternare pari e dispari

Obiettivo. Distinguere il numero di cambiamenti dalla lunghezza di una sequenza.

Leggi `n`, intero tra 1 e 20, poi `n` interi tra 0 e 50 nell'array `v`. Zero è pari. Le funzioni di questa coppia non devono modificare l'array.

A Versione base

Scrivi e usa `cambi_parita`, che riceve `n` e l'array e restituisce quante coppie di elementi adiacenti hanno parità diversa. Ogni coppia è formata da un elemento e da quello immediatamente successivo. Con un solo elemento il conteggio è zero. Stampa il risultato dopo `Cambi:`.

Input	Output
7 2 5 8 3 7 4 1	`Cambi: 5`
3 2 4 6	`Cambi: 0`

B Versione avanzata

Scrivi e usa `alternanza_massima`, che riceve `n` e l'array e restituisce la lunghezza della più lunga sequenza contigua in cui pari e dispari si alternano.

Due elementi consecutivi con la stessa parità interrompono l'alternanza. La lunghezza conta i valori, non le coppie. Un solo elemento costituisce un'alternanza di lunghezza 1. Stampa il risultato dopo `Lunghezza:`.

Input	Output
7 2 5 8 3 7 4 1	`Lunghezza: 4`
6 2 4 7 10 13 16	`Lunghezza: 5`
3 1 3 5	`Lunghezza: 1`

Casi da verificare. Un solo valore, tutti pari o tutti dispari, alternanza completa e sequenza più lunga alla fine.

10 Minimi e conche

Obiettivo. Esaminare i vicini e riconoscere gruppi di valori uguali consecutivi.

Leggi `n`, intero tra 1 e 20, poi `n` interi tra -20 e 20 nell'array `v`. Le funzioni di questa coppia non devono modificare l'array.

A Versione base

Un minimo locale è un elemento strettamente minore sia del valore immediatamente precedente sia di quello immediatamente successivo. Il primo e l'ultimo elemento non sono minimi locali.

Scrivi e usa `conta_minimi_locali`, che riceve `n` e l'array e restituisce quanti minimi locali sono presenti. Con meno di tre elementi il risultato è zero. Stampa il risultato dopo `Minimi:`.

Input	Output
5 4 1 2 0 5	`Minimi: 2`
4 3 1 1 4	`Minimi: 0`

B Versione avanzata

Considera ciascun blocco di valori uguali consecutivi, comprendendone tutte le ripetizioni adiacenti. Il blocco forma una conca se esistono un valore immediatamente prima e uno immediatamente dopo il blocco ed entrambi sono strettamente maggiori del valore comune del blocco.

Un blocco che inizia nella prima posizione o termina nell'ultima non è una conca. Una conca può essere composta da un solo elemento. Conta ogni conca una sola volta, anche quando contiene più elementi; conche separate si contano entrambe anche se hanno lo stesso valore.

Scrivi e usa `conta_conche`, che riceve `n` e l'array e restituisce il numero di conche. Stampa il risultato dopo `Conche:`.

Input	Output
7 4 1 1 3 0 0 5	`Conche: 2`
5 1 1 4 2 2	`Conche: 0`
1 0	`Conche: 0`

Casi da verificare. Blocchi ai bordi, array tutto uguale, conche di un solo elemento e due conche dello stesso valore.